Compiler Design
BCSE307L
Case Study

SQL QUERY COMPILER

SaiKaushick
Dev Nambiar
Sahasrad

Abstract:

Modern database systems process SQL queries through several stages that are very similar to the phases of a compiler. The main objective of this project is to design and implement a SQL Query Compiler that converts SQL queries into optimized execution plans for efficient processing.

The system accepts SQL queries containing common clauses such as SELECT, FROM, WHERE, AND, OR, and ORDER BY. When a query is entered, the lexical analyzer scans the input and breaks it into meaningful tokens such as keywords, identifiers, operators, and constants. These tokens are then passed to the parser, which checks whether the query follows the correct SQL grammar and generates a parse tree representing its structure.

After successful parsing, the semantic analyzer validates the query by checking the correctness of table names, column names, and other references. It ensures that the query is meaningful and can be executed without semantic errors. Once the query is validated, it is transformed into a relational algebra representation, which serves as the intermediate representation of the query.

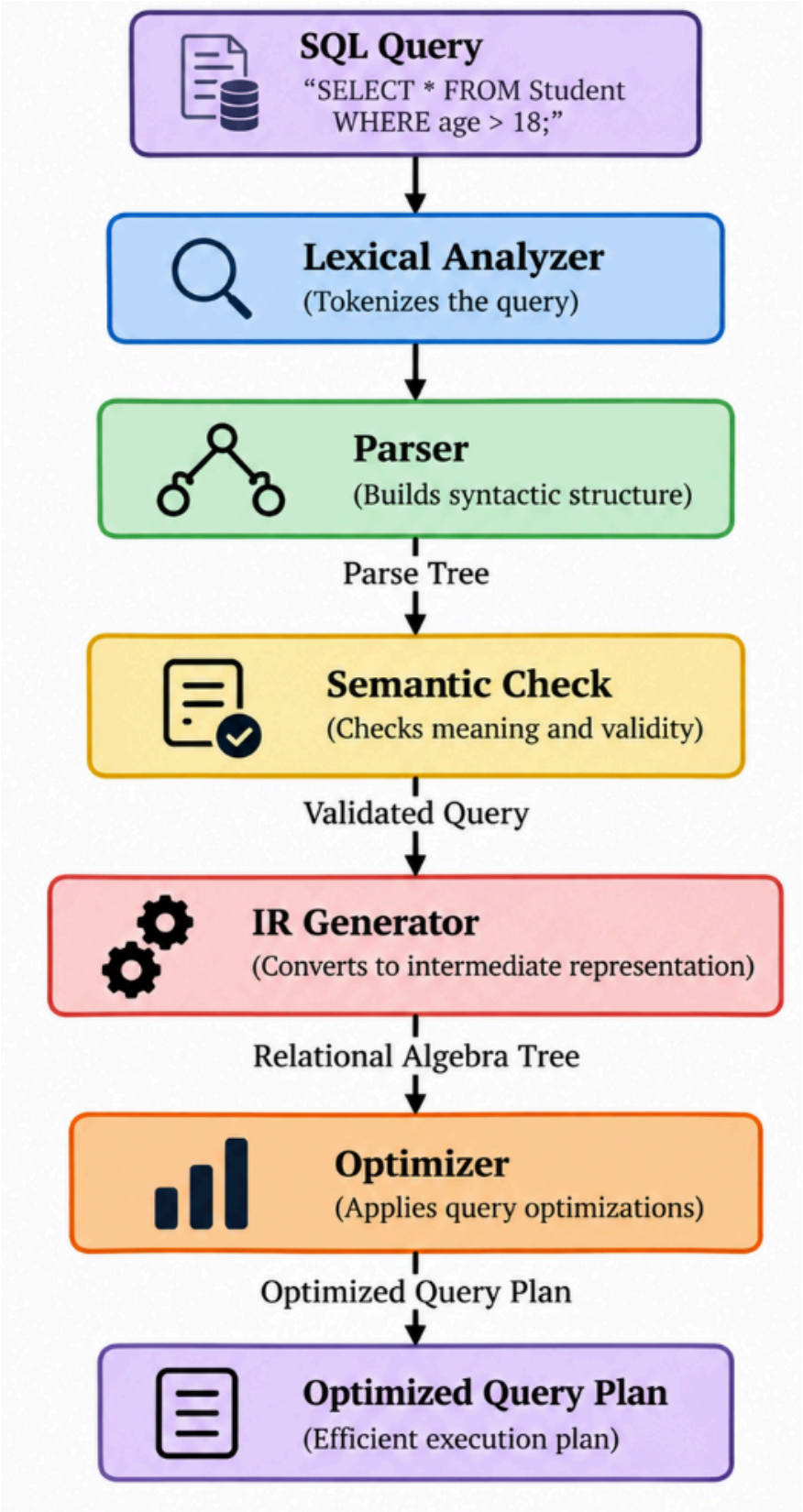
The next stage focuses on optimization. Query optimization techniques such as predicate pushdown and projection pushdown are applied to reduce unnecessary operations and improve execution efficiency. These optimizations help minimize the amount of data processed and lead to faster query execution. Based on the optimized representation, the system generates an optimized query plan that can be used by a database engine for execution.

Through this project, students can understand how fundamental compiler concepts such as lexical analysis, syntax analysis, semantic checking, intermediate representation generation, and optimization are applied in real-world database systems. The project demonstrates the strong relationship between compiler design and database query processing while providing practical experience in implementing different phases of a compiler-based system.

Architecture of the SQL Query Compiler:

1. **SQL Query Input**
    - The user provides an SQL query containing clauses such as `SELECT`, `FROM`, `WHERE`, and `ORDER BY etc`.
2. **Lexical Analyzer**
    - The lexical analyzer scans the query and breaks it into smaller units called tokens.
    - Examples of tokens include keywords, identifiers, operators, and constants.
3. **Parser**
    - The parser receives the tokens and checks whether the query follows the correct SQL grammar.
    - If the syntax is valid, it generates a Parse Tree that represents the structure of the query.
4. **Semantic Analyzer**
    - The semantic analyzer validates the query by checking table names, column names, and data references.
    - It ensures that the query is meaningful and free from semantic errors.

    - The output is a Validated Query.
5. **Intermediate Representation (IR) Generator**
    - The validated query is converted into a Relational Algebra Tree, which serves as the intermediate representation of the query.
    - This representation is easier to analyze and optimize.
6. **Query Optimizer**
    - The optimizer applies optimization techniques such as Predicate Pushdown (filtering unwanted rows as early as possible) and Projection Pushdown (means selecting only the required columns as early as possible).
    - These optimizations reduce unnecessary operations and improve query execution efficiency.
7. **Optimized Query Plan**
    - The final output is an optimized query execution plan that can be executed efficiently by a database system.


SQL Query
"SELECT * FROM Student
WHERE age > 18;"
Lexical Analyzer
(Tokenizes the query)
Parser
(Builds syntactic structure)
Parse Tree
Semantic Check
(Checks meaning and validity)
Validated Query
IR Generator
(Converts to intermediate representation)
Relational Algebra Tree
Optimizer
(Applies query optimizations)
Optimized Query Plan
Optimized Query Plan
(Efficient execution plan)

Proposed Article :

Base Paper: Low-Latency Query Compilation
Authors: Henning Funke, Jan Mühlig, Jens Teubner
Published in: The VLDB Journal, 2022

Relevance to the Project:

The paper explains how SQL queries can be converted into efficient execution plans using a query compilation process. It describes different stages such as query analysis, intermediate representation generation, and query optimization before execution.

These ideas are similar to the objectives of this project, where SQL queries pass through stages such as lexical analysis, parsing, semantic checking, intermediate representation generation, and optimization.

Inspired by this approach, the proposed SQL Query Compiler transforms SQL queries into relational algebra representations and applies optimization techniques to produce more efficient query execution plans.

Supporting Reference papers: *Access Path Selection in a Relational Database Management System*

Authors: Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G.

Published In: ACM SIGMOD Conference, 1979

Novelty:

Most compiler design projects focus on programming languages and code generation. This project is different because it applies compiler concepts to SQL query processing in a database environment.

The key features of the project are:

- Applying compiler phases such as lexical analysis, parsing, semantic checking, intermediate representation generation, and optimization to SQL queries.
- Using Relational Algebra Trees as the intermediate representation of the query.
- Converting SQL queries into query plans and showing how the database internally processes them.
- Implementing query optimization techniques to improve the efficiency of query execution.
- Displaying both the original query plan and the optimized query plan so that users can clearly see the impact of optimization.
- Demonstrating how compiler concepts can be used not only for programming languages but also for database query processing.

This project helps bridge the concepts of Compiler Design and Database Systems by showing how SQL queries can be analyzed, represented, and optimized using compiler techniques.